

1. Save & Load Docker Images

- We can use save & load docker images, if we want to export Docker images as tar files to move images across systems or save for future use.

- Helpful for sharing images without needing a registry.

Save an Image:

```
docker save -o <filename>.tar <image_name>
```

Example: `docker save -o my_image.tar my_app_image`

To load an Image from tar file:

```
docker load -i <filename>.tar
```

Example: `docker load -i my_image.tar`

2. Docker Top

Docker top command is used to list currently running processes inside a Docker container

Usage:

```
docker top <container_id or container_name>
```

Example: `docker top my_container`

3. Docker Pause & Unpause

Using pause & unpause, we can temporarily suspends or resume all processes in a container.

Useful during maintenance or when pausing activity without shutting down the container.

- To Pause a Container:

```
docker pause <container_id or container_name>
```

- Unpause a Container:

```
docker unpause <container_id or container_name>
```

4. Difference Between docker stop & docker pause

Feature	docker stop	docker pause
Why its used	To properly stop a running container	Temporarily freeze the Process running inside the container without stopping it
What it does	Completely shuts down the container	Freezes the container's running processes (like pause button)
When to use	When you want to stop the container fully	When you want to pause the container for some time (e.g., maintenance or testing)
To resume	No need – container is already stopped	Use docker unpause to continue

5. docker stats

Docker stats is used to check containers resource usage like CPU, memory, I/O of running containers in real time.

Usage:

```
docker stats <container_id or container_name>
```

Displays:

- CPU %: CPU usage
- MEM USAGE / LIMIT: Memory used / total
- NET I/O: Network input/output
- BLOCK I/O: Disk input/output
- PIDS: Number of processes

6. Docker Prune & System Disk Usage

Purpose: When docker host runs out of resources (i.e, disk full).

These Helps clean up system space by removing unused Docker resources like stopped containers, networks, dangling images, etc.

Clean up unused objects:

```
docker system prune
```

```
docker system prune -f
```

```
docker system prune --volumes
```

Show disk space used by docker:

```
docker system df
```

7. Docker Home Directory

Purpose: To know where Docker stores its internal data like images, containers, volumes, etc.

Useful for backups and troubleshooting.

Path: /var/lib/docker

8. ARG Instruction in Dockerfile

If we want to make Dockerfiles dynamic by allowing passing variables at build time.

Useful for customizing image builds.

Dockerfile Example:

```
ARG VERSION=latest
```

```
FROM ubuntu:${VERSION}
```

Build Command:

```
docker build --build-arg VERSION=20.04 -t myimage .
```

9. --rm in docker run

Purpose: it is an option that we can use with docker run command.

which will automatically removes the container once it stops, so that we need not to delete the container manually when a container is stopped.

this helps us Keeps system clean by avoiding clutter from temporary containers.

Usage:

```
docker run --rm <image> <command>
```

Example:

```
docker run --rm busybox echo "Hello!"
```

10. Get Container IP Address

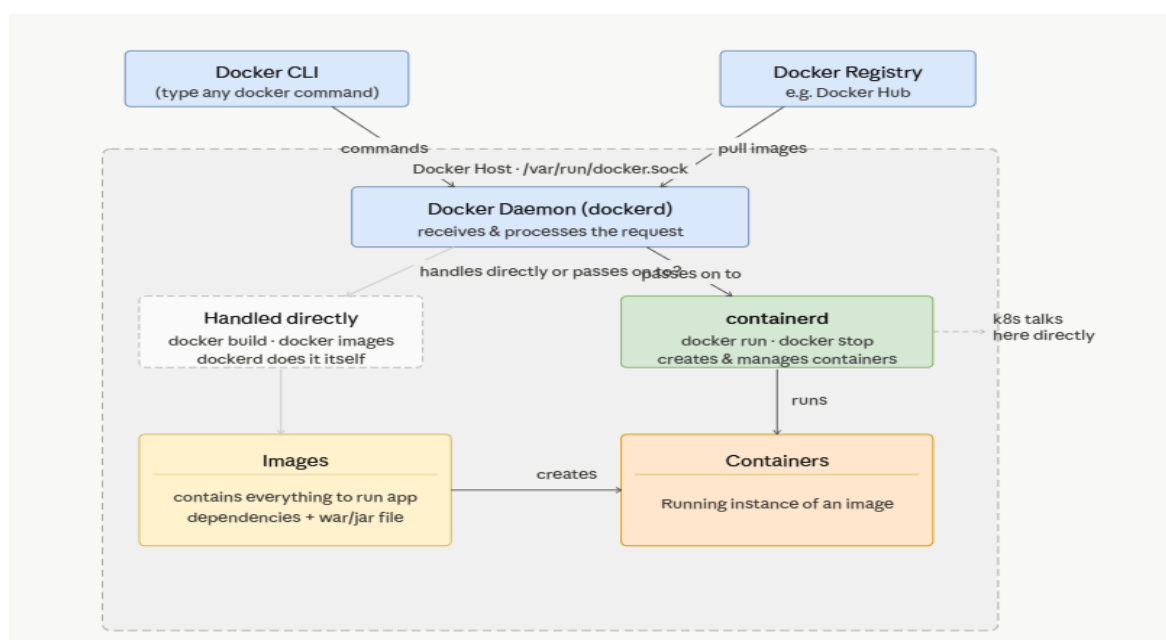
Purpose: To access services running inside the container or for network troubleshooting.

Methods:

1. Use the docker inspect command
2. Use ip addr command from inside the container
3. Use 'cat /etc/hosts' and check the entry for the container

11. Docker Architecture Overview:

Docker's architecture is designed to provide lightweight, portable, and self-sufficient containers that run applications consistently across different environments.



Flow of Docker Commands Work

1. When you run a Docker command (like `docker build` or `docker run`),
The CLI sends that API request to the Docker daemon (`dockerd`) via the Docker socket (`/var/run/docker.sock`).

The Docker daemon receives and processes the request:

2. The Docker daemon receives and processes the request:
 - For commands like `docker build` or `docker images` — `dockerd` handles it directly.
 - For container lifecycle commands like `docker run` or `docker stop` — `dockerd` passes these requests to `containerd`, which does the actual work of creating and managing the container.
3. Then the daemon sends the result back to the CLI, which displays it in your terminal.

Example: What happens step by step when you run `docker run nginx`?

1. You type `docker run nginx` in the terminal
2. Docker CLI sends an API request to `dockerd` via `/var/run/docker.sock`
3. `dockerd` checks if the `nginx` image is available locally
4. If not, it pulls the image from Docker Hub
5. `dockerd` passes the request to create the container to `containerd`
6. `containerd` creates and starts the container
7. The running container is a live instance of the `nginx` image

12. What is the Docker Socket?

- The **Docker socket** (`/var/run/docker.sock`) is a Unix domain socket.
- It acts as a communication endpoint between the Docker client and the Docker daemon.
- When you run a Docker command, the client communicates with the daemon through this socket.

13. Why docker socket is important?

- It allows Docker commands to control the daemon directly.
- It can be mounted inside containers to give containers control over Docker on the host (which can be risky if not managed properly).
-

14. What if docker goes down?

Docker service may go down due to below reasons

- ❖ **Disk is full** – No free space on system, so Docker can't run or save anything.
- ❖ **RAM is low** – If system memory is full, OS may stop Docker to save itself.
- ❖ **Wrong config file** – Mistake in /etc/docker/daemon.json stops Docker from starting.
- ❖ **Containerd not running** – Docker depends on containerd; if it's down, Docker won't work.
- ❖ **Socket file issue** – Docker CLI can't talk to the daemon if /var/run/docker.sock is missing or broken.

```
ubuntu@dockerHost:~$ sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Mon 2025-06-23 10:23:21 UTC; 2h 39min ago
   TriggeredBy: ● docker.socket
   Docs: https://docs.docker.com
   Main PID: 2489 (dockerd)
   Tasks: 11
   Memory: 149.0M (peak: 217.0M)
   CPU: 2.881s
   CGroup: /system.slice/docker.service
           └─2489 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

Jun 23 10:23:21 ip-172-31-3-139 dockerd[2489]: time="2025-06-23T10:23:21.204105259Z" level=info msg="Loading containers: start."
Jun 23 10:23:21 ip-172-31-3-139 dockerd[2489]: time="2025-06-23T10:23:21.537932643Z" level=info msg="Loading containers: done."
Jun 23 10:23:21 ip-172-31-3-139 dockerd[2489]: time="2025-06-23T10:23:21.559347630Z" level=info msg="Docker daemon" commit=45873be containerd-snapshotter=false storage-driver=overlayfs2
Jun 23 10:23:21 ip-172-31-3-139 dockerd[2489]: time="2025-06-23T10:23:21.559416166Z" level=info msg="Initializing buildkit"
Jun 23 10:23:21 ip-172-31-3-139 dockerd[2489]: time="2025-06-23T10:23:21.575304718Z" level=warning msg="CDI setup error /var/run/cdi: failed to monitor for changes: no such file or directory"
Jun 23 10:23:21 ip-172-31-3-139 dockerd[2489]: time="2025-06-23T10:23:21.575327623Z" level=warning msg="CDI setup error /etc/cdi: failed to monitor for changes: no such file or directory"
Jun 23 10:23:21 ip-172-31-3-139 dockerd[2489]: time="2025-06-23T10:23:21.614674284Z" level=info msg="Completed buildkit initialization"
Jun 23 10:23:21 ip-172-31-3-139 dockerd[2489]: time="2025-06-23T10:23:21.628411534Z" level=info msg="Daemon has completed initialization"
Jun 23 10:23:21 ip-172-31-3-139 dockerd[2489]: time="2025-06-23T10:23:21.628652987Z" level=info msg="API listen on /run/docker.sock"
Jun 23 10:23:21 ip-172-31-3-139 systemd[1]: Started docker.service - Docker Application Container Engine.
ubuntu@dockerHost:~$
```

Explanation:

- **Service name:** docker.service manages Docker on the system.
- **Status:** Active: active (running) → Docker is running fine.
- **Enabled:** Docker starts automatically at system boot.
- **Active: active (running):** Docker is currently running.
- **Main Process:** Process ID is 2489, which is the main Docker process.
- **Memory/CPU:** Shows how much system resource Docker is using.
- **TriggeredBy: docker.socket:** Docker listens on the socket /var/run/docker.sock.
- **docker.socket:** The communication point for Docker commands.
- **Logs:** Recent logs show daemon startup, container loading, etc.

15. How to troubleshoot or fix if docker goes down?

Step	Command	Observation
Check Docker status	<code>sudo systemctl status docker</code>	Confirms if service is running or failed.
Try starting it manually	<code>sudo systemctl start docker</code>	Starts the Docker daemon manually.
Check why it failed	<code>journalctl -xeu docker</code>	Shows detailed error logs for Docker service.
Restart the service	<code>sudo systemctl restart docker</code>	Fixes temporary issues like socket or memory error.
Check Docker socket	<code>ls -l /var/run/docker.sock</code>	If socket file is missing or wrong permission, Docker CLI can't connect.
Check dockerhost disk space	<code>df -kh</code>	If disk is full, Docker won't start or load images.